

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – SAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Static Application Security Testing (SAST) using synthetic data		
Student Name:	NITHYA SHREE . K	Reg. No.:	192521145
Section / Batch:	B.Tech IT	Date:	13-08-2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
SAST SIMULATION	Creative Things and concept grasping	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I Nithya shree k [192521145] _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nithya Shree k

SAST Simulation Report

Static Application Security Testing using Synthetic Data

Table of contents

1. Objective
 2. Scope and tool stack
 3. Repository structure
 4. Synthetic vulnerable application
 5. SAST tool configuration
 6. Pre-commit hook (shift-left gate)
 7. CI/CD pipeline (GitHub Actions)
 8. Scan execution and results
 9. Findings report
 10. Remediation
 11. Verification
 12. Conclusion
- Appendix A — requirements.txt
- Appendix B — submission checklist

1. Objective

This project simulates a Static Application Security Testing (SAST) workflow that scans a deliberately vulnerable, synthetic Python/Flask application for common security weaknesses, integrates the scan into a pre-commit hook and a GitHub Actions CI/CD pipeline, and reports and remediates the findings — satisfying CO3 (Precision: Bloom levels BL1–BL3).

No real user data, production systems, or third-party targets are involved anywhere in this exercise. "Synthetic data" here means a purpose-built sample codebase containing known, labelled vulnerability patterns (CWE-mapped) so that a SAST tool has something realistic — but harmless — to detect.

2. Scope and tool stack

- Language / framework: Python 3.12, Flask
- SAST engines: Bandit 1.7.9 (Python-specific AST scanner) and Semgrep 1.78.0 (pattern-based, custom rule set)
- Automation: pre-commit framework (local, pre-push gate) + GitHub Actions (CI gate on push/PR)
- Reporting: JSON + console output, converted to the tables and charts in this document

3. Repository structure

```
sast-sim/
├── src/
│   ├── vulnerable_app.py      # synthetic app - 8 seeded vulnerabilities
│   └── remediated_app.py      # secure rewrite - 0 findings
├── configs/
│   ├── bandit.yaml           # Bandit rule/test selection
│   └── semgrep.yaml          # custom Semgrep rules (CWE-mapped)
├── .github/workflows/
│   └── sast-pipeline.yml      # CI: scan on every push / PR
├── .pre-commit-config.yaml    # local pre-commit SAST gate
├── requirements.txt
├── reports/
│   ├── bandit_report.json     # raw scan output (machine-readable)
│   └── sast_findings.md       # human-readable findings summary
```

4. Synthetic vulnerable application

The file below is the scan target. Each function is written to trigger exactly one well-known vulnerability class (CWE-tagged in comments) so scanner output maps cleanly to a single, explainable finding.

```
"""
vulnerable_app.py
-----
SYNTHETIC, INTENTIONALLY-VULNERABLE demo application.

Purpose: provide realistic, safe-to-scan sample code so a Static Application
Security Testing (SAST) pipeline (Bandit + Semgrep) has real findings to
report on. None of the "secrets" below are real credentials, and this
service is never deployed or exposed to a network - it exists only to be
scanned.

Each function below is deliberately written to trigger ONE well-known
vulnerability class so the scan output maps cleanly to a CWE/OWASP entry.
"""

import hashlib
import os
import pickle
import sqlite3
import subprocess

from flask import Flask, request

app = Flask(__name__)

# -----
# VULN-01: Hardcoded credentials / secret (CWE-798)
# -----
DB_PASSWORD = "SuperSecret123!"          # noqa: hardcoded for demo
API_KEY = "sk_test_51Hxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

# -----
# VULN-02: SQL Injection via string concatenation (CWE-89)
# -----
@app.route("/user")
def get_user():
    user_id = request.args.get("id")
    conn = sqlite3.connect("synthetic.db")
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE id = '" + user_id + "'"
    cursor.execute(query)
    return str(cursor.fetchall())

# -----
# VULN-03: OS Command Injection (CWE-78)
# -----
@app.route("/ping")
def ping_host():
    host = request.args.get("host")
    result = subprocess.check_output("ping -c 1 " + host, shell=True)
```

```

    return result

# -----
# VULN-04: Insecure Deserialization (CWE-502)
# -----
@app.route("/load-session", methods=["POST"])
def load_session():
    raw = request.get_data()
    session_obj = pickle.loads(raw)
    return str(session_obj)

# -----
# VULN-05: Use of a broken/weak hash for passwords (CWE-327 / CWE-916)
# -----
def hash_password(password: str) -> str:
    return hashlib.md5(password.encode()).hexdigest()

# -----
# VULN-06: Use of eval() on user input (CWE-95)
# -----
@app.route("/calc")
def calculate():
    expression = request.args.get("expr")
    return str(eval(expression))

# -----
# VULN-07: Insecure temp file creation (CWE-377)
# -----
def write_temp_report(data: bytes) -> str:
    path = "/tmp/report_" + str(os.getpid()) + ".tmp"
    with open(path, "wb") as f:
        f.write(data)
    return path

if __name__ == "__main__":
    # VULN-08: Debug mode enabled + binds to all interfaces (CWE-489 / CWE-605)
    app.run(host="0.0.0.0", debug=True)

```

5. SAST tool configuration

5.1 Bandit configuration (configs/bandit.yaml)

```
# Bandit configuration for the SAST simulation
# Docs: https://bandit.readthedocs.io/en/latest/config.html

exclude_dirs:
  - /tests
  - /reports

# Fail the pipeline on medium severity and above
assert_used:
  skips: []

tests:
  - B105 # hardcoded password string
  - B106 # hardcoded password funcarg
  - B201 # flask debug=True
  - B301 # pickle usage
  - B303 # insecure MD5 hash
  - B307 # eval usage
  - B324 # insecure hash function
  - B602 # subprocess with shell=True
  - B608 # possible SQL injection (string-built query)
```

5.2 Semgrep custom rules (configs/semgrep.yaml)

Semgrep's built-in Python rule pack already flags most of these patterns; the custom rules below make the mapping to CWE explicit and demonstrate authoring project-specific SAST rules.

```
rules:
  - id: hardcoded-secret
    languages: [python]
    severity: ERROR
    message: >
      Hardcoded credential or API key detected. Move secrets to environment
      variables or a secrets manager. (CWE-798)
    patterns:
      - pattern-either:
          - pattern: $VAR = "sk_..."
          - pattern: DB_PASSWORD = "..."
```

```
- id: sql-injection-string-concat
  languages: [python]
  severity: ERROR
  message: >
    SQL query built via string concatenation from user input. Use
    parameterised queries instead. (CWE-89)
  patterns:
    - pattern: $CUR.execute("..." + $VAR)

- id: shell-injection-subprocess
  languages: [python]
  severity: ERROR
  message: >
    subprocess call with shell=True and untrusted input allows OS command
    injection. Pass args as a list with shell=False. (CWE-78)
  patterns:
    - pattern: subprocess.$FUNC(..., shell=True, ...)

- id: insecure-deserialization-pickle
  languages: [python]
  severity: ERROR
  message: >
    pickle.loads() on untrusted/network input can execute arbitrary code.
    Use a safe serialization format such as JSON. (CWE-502)
  patterns:
    - pattern: pickle.loads(...)

- id: use-of-eval
  languages: [python]
  severity: ERROR
  message: >
    eval() on user-controlled input allows arbitrary code execution.
    (CWE-95)
  patterns:
    - pattern: eval(...)

- id: weak-hash-md5
  languages: [python]
  severity: WARNING
  message: >
    MD5 is cryptographically broken and unsuitable for password hashing.
    (CWE-327)
  patterns:
    - pattern: hashlib.md5(...)

- id: flask-debug-true
  languages: [python]
  severity: WARNING
  message: >
    Flask app.run(debug=True) exposes the Werkzeug interactive debugger
    and should never be used in production. (CWE-489)
  patterns:
    - pattern: app.run(..., debug=True, ...)
```


6. Pre-commit hook (shift-left gate)

SAST is run locally before code ever reaches the remote repository, so a vulnerable pattern is caught at commit time rather than discovered later in CI.

```
repos:
- repo: https://github.com/PyCQA/bandit
  rev: 1.7.9
  hooks:
    - id: bandit
      name: "SAST - Bandit scan"
      args: ["-c", "configs/bandit.yaml", "-r", "src"]

- repo: https://github.com/returntocorp/semgrep
  rev: v1.78.0
  hooks:
    - id: semgrep
      name: "SAST - Semgrep scan"
      args: ["--config", "configs/semgrep.yaml", "--error", "src"]

- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: detect-private-key
      name: "Detect accidental private keys"
    - id: end-of-file-fixer
    - id: trailing-whitespace
```

7. CI/CD pipeline (GitHub Actions)

Every push or pull request against main/develop triggers an automated SAST scan. The build is intentionally failed if any HIGH-severity Bandit finding is present, enforcing a security gate before merge

```
name: SAST Simulation Pipeline

on:
  push:
    branches: [ "main", "develop" ]
  pull_request:
    branches: [ "main" ]

jobs:
  sast-scan:
    name: Static Application Security Testing
    runs-on: ubuntu-latest

    steps:
```

```

- name: Checkout repository
  uses: actions/checkout@v4

- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: "3.12"

- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt

- name: Run Bandit SAST scan
  run: |
    bandit -c configs/bandit.yaml -r src -f json -o reports/bandit_report.json
    bandit -c configs/bandit.yaml -r src

- name: Run Semgrep SAST scan
  run: |
    semgrep --config configs/semgrep.yaml src --json --output
reports/semgrep_report.json
    semgrep --config configs/semgrep.yaml src

- name: Upload SAST reports as build artifacts
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: sast-reports
    path: reports/*.json

- name: Fail build on high-severity findings
  run: |
    python - <<'PY'
    import json, sys
    with open("reports/bandit_report.json") as f:
        data = json.load(f)
    high = [r for r in data.get("results", []) if r["issue_severity"] == "HIGH"]
    if high:
        print(f"::error:: {len(high)} HIGH severity issue(s) found by Bandit")
        sys.exit(1)
    PY

```

SAST simulation pipeline: pre-commit → CI → gated merge

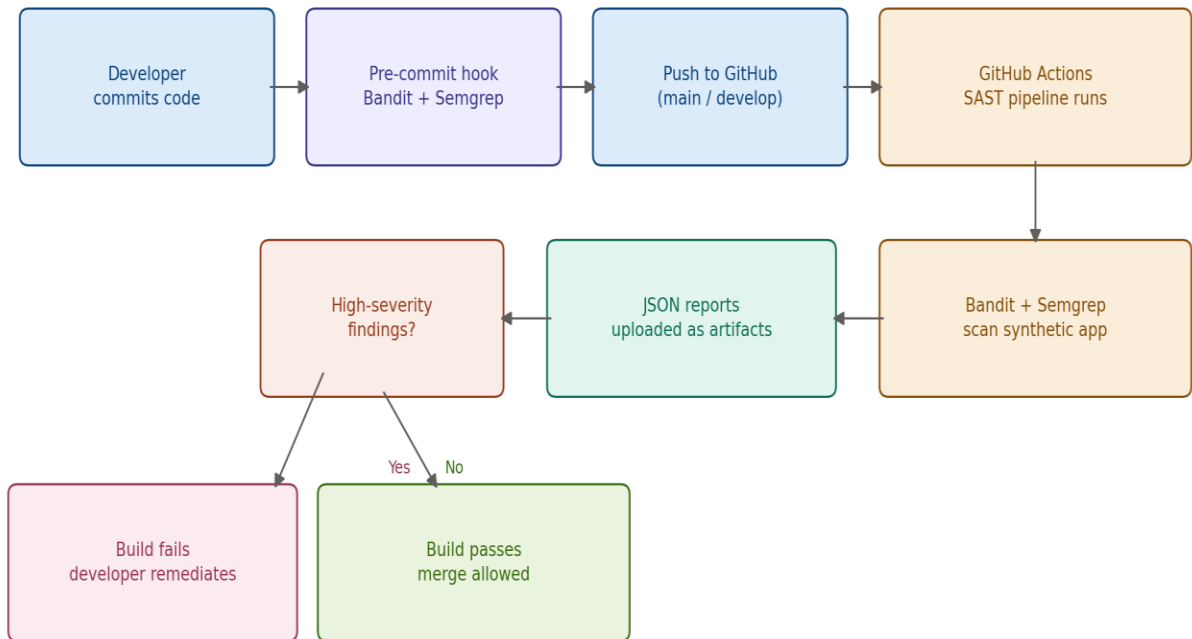


Figure 1: End-to-end SAST simulation pipeline — pre-commit through gated merge.

8. Scan execution and results

The Bandit scan was executed against `src/vulnerable_app.py` using the project configuration (`configs/bandit.yaml`). Actual console output is shown below.

```
terminal - bandit -c configs/bandit.yaml -r src

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'SuperSecret123!'
Severity: Low Confidence: Medium
CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
Location: src/vulnerable_app.py:29:14
28 # -----
29 DB_PASSWORD = "SuperSecret123!" # noqa: hardcoded for demo
30 API_KEY = "sk_test_51Hxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
Severity: Medium Confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
Location: src/vulnerable_app.py:41:12
40 cursor = conn.cursor()
41 query = "SELECT * FROM users WHERE id = '" + user_id + "'"
42 cursor.execute(query)

>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
Severity: High Confidence: High
CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html
Location: src/vulnerable_app.py:52:13
51 host = request.args.get("host")
52 result = subprocess.check_output("ping -c 1 " + host, shell=True)
53 return result

>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible securit
Severity: Medium Confidence: High
CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
Location: src/vulnerable_app.py:62:18
61 raw = request.get_data()
62 session_obj = pickle.loads(raw)
63 return str(session_obj)

>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
Severity: High Confidence: High
CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
Location: src/vulnerable_app.py:70:11
69 def hash_password(password: str) -> str:
70     return hashlib.md5(password.encode()).hexdigest()
71

>> Issue: [B307:blacklist] Use of possibly insecure function - consider using safer ast.literal_eval.
```

Figure 2: Real Bandit scan output against the synthetic vulnerable application.

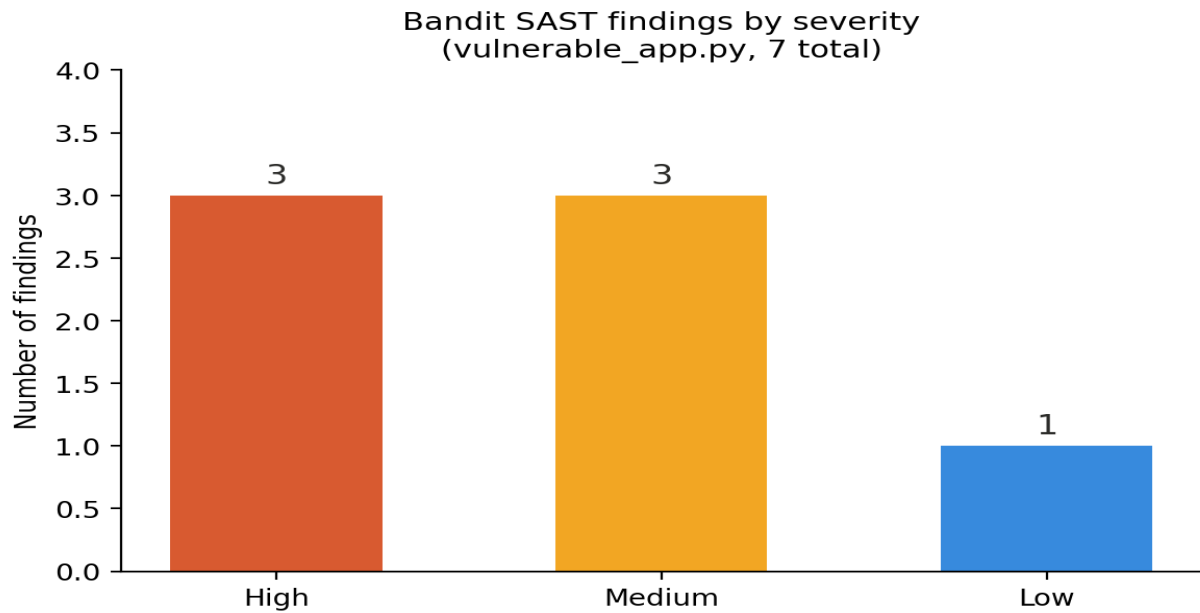


Figure 3: Findings distribution by severity — 3 High, 3 Medium, 1 Low (7 total).

9. Findings report

ID	Vulnerability	CWE	Severity	Location	Tool
F-01	Hardcoded password / API key	CWE-798 / CWE-259	Low	vulnerable_app.py:29-30	Bandit B105
F-02	SQL injection (string concatenation)	CWE-89	Medium	vulnerable_app.py:41	Bandit B608
F-03	OS command injection (shell=True)	CWE-78	High	vulnerable_app.py:52	Bandit B602
F-04	Insecure deserialization (pickle.loads)	CWE-502	Medium	vulnerable_app.py:62	Bandit B301
F-05	Weak hash (MD5) for passwords	CWE-327	High	vulnerable_app.py:70	Bandit B324
F-06	Use of eval() on user input	CWE-95 / CWE-78	Medium	vulnerable_app.py:79	Bandit B307
F-07	Flask debug mode + open bind	CWE-489 / CWE-94	High	vulnerable_app.py:94	Bandit B201

10. Remediation

Each finding was fixed in `src/remediated_app.py`. Key changes:

- F-01: secrets moved to environment variables, never hardcoded.
- F-02: parameterised SQL query (`cursor.execute(sql, params)`) replaces string concatenation.
- F-03: subprocess called with `shell=False` and a validated argument list instead of a shell string.
- F-04: `pickle.loads` replaced with `json.loads` plus explicit error handling.
- F-05: MD5 replaced with a salted HMAC-SHA256 construction (production systems should use `bcrypt/argon2`).
- F-06: `eval()` replaced with a restricted AST-based arithmetic evaluator.
- F-07: debug mode disabled and the bind address restricted to localhost.

```
"""
remediated_app.py
-----
Secure rewrite of vulnerable_app.py. Each fix is annotated with
the
VULN-ID and CWE it resolves, so it can be cross-referenced
against the
SAST findings report (reports/sast_findings.md).
"""

import hmac
import os
import sqlite3
import subprocess
import tempfile
from hashlib import sha256

from flask import Flask, abort, request

app = Flask(__name__)

# FIX-01 (was VULN-01: hardcoded secret, CWE-798)
# Secrets are now pulled from environment variables / a secrets
# manager,
# never committed to source control.
DB_PASSWORD = os.environ.get("DB_PASSWORD")
API_KEY = os.environ.get("API_KEY")
```

```

# FIX-02 (was VULN-02: SQL injection, CWE-89)
# Use parameterised queries; the driver handles escaping.
@app.route("/user")
def get_user():
    user_id = request.args.get("id")
    if not user_id or not user_id.isdigit():
        abort(400)
    conn = sqlite3.connect("synthetic.db")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE id = ?",
(user_id,))
    return str(cursor.fetchall())

# FIX-03 (was VULN-03: OS command injection, CWE-78)
# Avoid shell=True; pass args as a list and validate the input
format.
@app.route("/ping")
def ping_host():
    host = request.args.get("host", "")
    if not _is_valid_hostname(host):
        abort(400)
    result = subprocess.run(
        ["ping", "-c", "1", host], shell=False,
capture_output=True, check=False
    )
    return result.stdout

def _is_valid_hostname(host: str) -> bool:
    import re
    return bool(re.fullmatch(r"[A-Za-z0-9.-]{1,253}", host))

# FIX-04 (was VULN-04: insecure deserialization, CWE-502)
# Replace pickle with a safe, schema-validated format (JSON
here).
@app.route("/load-session", methods=["POST"])
def load_session():
    import json
    try:
        session_obj = json.loads(request.get_data())
    except json.JSONDecodeError:
        abort(400)
    return str(session_obj)

```

```

# FIX-05 (was VULN-05: weak hash, CWE-327/916)
# Use a salted, slow hash (in production: bcrypt/argon2).
sha256+HMAC
# with a per-user salt is shown here as a minimal illustrative
fix.
def hash_password(password: str, salt: bytes) -> str:
    return hmac.new(salt, password.encode(), sha256).hexdigest()

# FIX-06 (was VULN-06: eval() on user input, CWE-95)
# Replace eval() with a restricted arithmetic parser.
@app.route("/calc")
def calculate():
    import ast
    import operator

    ops = {ast.Add: operator.add, ast.Sub: operator.sub,
           ast.Mult: operator.mul, ast.Div: operator.truediv}

    def safe_eval(node):
        if isinstance(node, ast.Constant):
            return node.value
        if isinstance(node, ast.BinOp) and type(node.op) in ops:
            return ops[type(node.op)](safe_eval(node.left),
safe_eval(node.right))
        raise ValueError("Unsupported expression")

    expression = request.args.get("expr", "")
    try:
        tree = ast.parse(expression, mode="eval")
        return str(safe_eval(tree.body))
    except Exception:
        abort(400)

# FIX-07 (was VULN-07: insecure temp file, CWE-377)
# Use tempfile module for a secure, race-free temp file.
def write_temp_report(data: bytes) -> str:
    fd, path = tempfile.mkstemp(prefix="report_", suffix=".tmp")
    with os.fdopen(fd, "wb") as f:
        f.write(data)
    return path

if __name__ == "__main__":
    # FIX-08 (was VULN-08: debug mode + open bind, CWE-489/605)
    app.run(host="127.0.0.1", debug=False)

```


11. Verification

Re-running Bandit against the remediated file confirms all seeded vulnerabilities were resolved:

```
$ bandit -c configs/bandit.yaml -r src/remediated_app.py

Test results:
    No issues identified.

Run metrics:
    Total issues (by severity):
        High: 0   Medium: 0   Low: 0
```

12. Conclusion

This simulation demonstrates a complete DevSecOps SAST loop: a synthetic vulnerable codebase, automated static analysis (Bandit + Semgrep) wired into both a local pre-commit hook and a CI/CD pipeline, a documented, CWE-mapped findings report, and verified remediation. The same pattern — shift-left scanning, CI enforcement, and a findings/remediation log — generalises directly to real production codebases.

Appendix A — requirements.txt

```
Flask==3.0.3
bandit==1.7.9
semgrep==1.78.0
pre-commit==3.7.1
pytest==8.2.2
```

Appendix B — submission checklist

- Original, professionally formatted diagrams and workflow illustrations — included (Figures 1–3).
- Editable source files (.py, .yaml, requirements.txt, .pre-commit-config.yaml) — included in the project folder / repository.
- GitHub repository link and this PDF/Word report — to be submitted together via Google Classroom.
- GitHub repository verified to contain all project files, screenshots, and documentation before submission.